



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

01 - Linear Regression/Least Squares

François Pitié

Assistant Professor in Media Signal Processing

Department of Electronic & Electrical Engineering, Trinity College Dublin

[4C16/5C16] Deep Learning and its Applications — 2026/2027

Introduction

We will start our journey into Machine Learning (ML) with a topic that should be familiar to you: [Linear Regression](#).

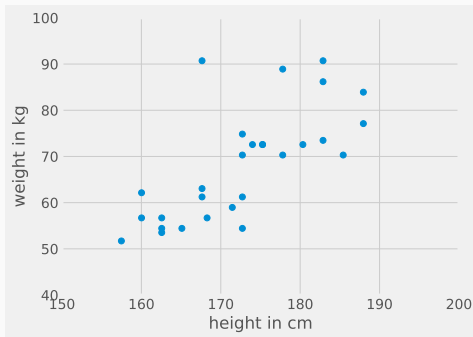
We will revisit this classical technique through the modern lens of Machine Learning.

Least Squares can be considered the original Machine Learning algorithm.

By examining it, we can introduce many of the fundamental concepts that form the bedrock of modern ML:

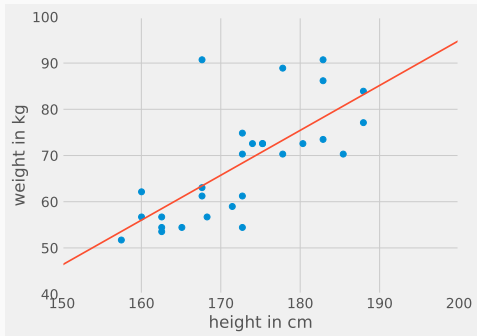
- training and testing data,
- overfitting and underfitting,
- regularisation,
- the modelling of noise and the concept of a loss function.

Linear Regression: Motivating Data



We have collected some data.

Linear Regression: Prediction Goal



We are looking to infer a linear prediction:

$$\text{weight}(\text{kg}) = \text{height}(\text{cm}) \times 0.972 - 99.5$$

Linear Regression: Model Definition

The **input** is a feature vector $(x_1, \dots, x_p)$.

The **output** is a scalar y . (You can easily generalise to a vector of output values by splitting the problem into as many sub-problems.)

Our **model** links the output y to the input feature vector $(x_1, \dots, x_p)$ with a linear relationship:

$$y = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_px_p$$

Linear Regression: Observations

In practice, you have n observations, for which you have extracted p features:

$$y_1 = w_0 + w_1x_{11} + w_2x_{12} + \cdots + w_px_{1p} + \varepsilon_1$$

$$y_2 = w_0 + w_1x_{21} + w_2x_{22} + \cdots + w_px_{2p} + \varepsilon_2$$

$$y_3 = w_0 + w_1x_{31} + w_2x_{32} + \cdots + w_px_{3p} + \varepsilon_3$$

$$\vdots$$

$$y_n = w_0 + w_1x_{n1} + w_2x_{n2} + \cdots + w_px_{np} + \varepsilon_n$$

As the model can't explain everything, we introduce an error term ε .

Linear Regression: Loss Function

We want to find the weights $w_0, w_1, \dots, w_p$ that minimise the error.

At this point, the error $(\varepsilon_i)_{1 \leq i \leq n}$ is a vector of n separate terms. Since we can't minimise a vector, we aggregate the values into a single scalar.

In Machine Learning, this aggregate error function is called a **loss function**.

In linear regression, we use the **mean squared error** (MSE) as the loss function:

$$E = \frac{1}{n} \sum_{i=1}^n \varepsilon_i^2 = \frac{1}{n} \sum_{i=1}^n (w_0 + w_1 x_{i1} + \dots + w_p x_{ip} - y_i)^2$$

The choice of MSE is a fundamental aspect of linear regression. Other loss functions are possible (e.g. mean absolute error) but lead to very different mathematics.

Derivation: Calculus Approach

Let's derive the values $w_0, w_1, \dots, w_p$ that minimise the mean squared error function $E(w_0, w_1, \dots, w_p)$.

Derivation: Setting Partial Derivatives to Zero

$$E(w_0, \dots, w_p) = \frac{1}{n} \sum_{i=1}^n (w_0 + w_1 x_{i1} + \dots + w_p x_{ip} - y_i)^2$$

At the minimum of E , we have $\frac{\partial E}{\partial w_0} = \dots = \frac{\partial E}{\partial w_p} = 0$:

$$\frac{\partial E}{\partial w_0}(w_0, \dots, w_p) = \frac{2}{n} \sum_{i=1}^n (w_0 + w_1 x_{i1} + \dots + w_p x_{ip} - y_i) = 0$$

$$\frac{\partial E}{\partial w_1}(w_0, \dots, w_p) = \frac{2}{n} \sum_{i=1}^n x_{i1} (w_0 + w_1 x_{i1} + \dots + w_p x_{ip} - y_i) = 0$$

$\vdots$

$$\frac{\partial E}{\partial w_p}(w_0, \dots, w_p) = \frac{2}{n} \sum_{i=1}^n x_{ip} (w_0 + w_1 x_{i1} + \dots + w_p x_{ip} - y_i) = 0$$

Derivation: The Linear System

Rearranging terms and dividing by $2/n$:

$$\begin{array}{ccccccc} w_0 \sum_{i=1}^n 1 & + w_1 \sum_{i=1}^n x_{i1} & + \dots + w_p \sum_{i=1}^n x_{ip} & = & \sum_{i=1}^n y_i \\ w_0 \sum_{i=1}^n x_{i1} + w_1 \sum_{i=1}^n x_{i1}^2 & + \dots + w_p \sum_{i=1}^n x_{i1} x_{ip} & = & \sum_{i=1}^n x_{i1} y_i \\ \vdots & \vdots & \vdots & & \vdots \\ w_0 \sum_{i=1}^n x_{ip} + w_1 \sum_{i=1}^n x_{ip} x_{i1} + \dots + w_p \sum_{i=1}^n x_{ip}^2 & = & \sum_{i=1}^n x_{ip} y_i \end{array}$$

This gives a linear system of $p+1$ equations, solvable efficiently using standard linear solvers.

Matrix Calculus

We are now going to derive the exact same equations using matrix notations. It is much cleaner and avoids having to track nested summations.

Matrix Notations: Definitions

By convention, we write a scalar as x , a vector as $\mathbf{x}$ and a matrix as $\mathbf{X}$.
We denote:

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad \mathbf{X} = \begin{pmatrix} 1 & x_{11} & \dots & x_{1p} \\ 1 & x_{21} & \dots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \dots & x_{np} \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_p \end{pmatrix}, \quad \boldsymbol{\varepsilon} = \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{pmatrix}$$

The linear model compactly becomes:

$$\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\varepsilon}$$

The matrix $\mathbf{X}$, stacking all observations, is called the [Design Matrix](#).

Matrix Notations: The Bias Term

Let us pause here for a second and revisit our scalar model definition:

$$y = w_0 + w_1x_1 + \cdots + w_px_p$$

Notice that w_0 plays a distinct role as an intercept: in machine learning and deep learning, this weight w_0 has been given a special name: **the bias**.

Matrix Notations: Augmented Features

In the rest of the course, we consider the bias w_0 as the weight of an augmented feature with constant value 1. Thus, the row feature vector for observation i is:

$$\mathbf{x}_i = \begin{pmatrix} 1 \\ x_{i1} \\ \vdots \\ x_{ip} \end{pmatrix}$$

The linear model is written as $y_i = \mathbf{x}_i^\top \mathbf{w}$, and the design matrix is:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{pmatrix}$$

Matrix Notations: Matrix Loss & Gradient

The mean squared error can be formulated in matrix notation as:

$$\begin{aligned} E(\mathbf{w}) &= \frac{1}{n} \sum_{i=1}^n \varepsilon_i^2 = \frac{1}{n} \boldsymbol{\varepsilon}^\top \boldsymbol{\varepsilon} = \frac{1}{n} \|\boldsymbol{\varepsilon}\|^2 \\ &= \frac{1}{n} (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) \\ &= \frac{1}{n} (\mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} + \mathbf{y}^\top \mathbf{y} - 2\mathbf{w}^\top \mathbf{X}^\top \mathbf{y}) \end{aligned}$$

At the minimum of $E(\mathbf{w})$, the gradient vector vanishes:

$$\frac{\partial E}{\partial \mathbf{w}} = \left(\frac{\partial E}{\partial w_0}, \dots, \frac{\partial E}{\partial w_p} \right)^\top = \mathbf{0}$$

$\frac{\partial E}{\partial \mathbf{w}}$ is the **gradient** of E , denoted as ∇E .

Gradient Calculus: Identities

Below is a reference list of useful matrix and vector gradient identities. Vectors $\mathbf{a}$, $\mathbf{b}$ and matrix $\mathbf{A}$ are independent of $\mathbf{w}$.

$$\frac{\partial \mathbf{a}^\top \mathbf{w}}{\partial \mathbf{w}} = \mathbf{a}$$

$$\frac{\partial \mathbf{b}^\top \mathbf{A} \mathbf{w}}{\partial \mathbf{w}} = \mathbf{A}^\top \mathbf{b}$$

$$\frac{\partial \mathbf{w}^\top \mathbf{A} \mathbf{w}}{\partial \mathbf{w}} = (\mathbf{A} + \mathbf{A}^\top) \mathbf{w} \quad (2\mathbf{A} \mathbf{w} \text{ if } \mathbf{A} \text{ is symmetric})$$

$$\frac{\partial \mathbf{w}^\top \mathbf{w}}{\partial \mathbf{w}} = 2\mathbf{w}$$

$$\frac{\partial (\mathbf{a}^\top \mathbf{w})(\mathbf{w}^\top \mathbf{b})}{\partial \mathbf{w}} = (\mathbf{a} \mathbf{b}^\top + \mathbf{b} \mathbf{a}^\top) \mathbf{w}$$

Exercise: Matrix Gradient

Exercise:

Compute the gradient $\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}}$ for

$$E(\mathbf{w}) = (\mathbf{w} - \mathbf{B}\mathbf{w})^\top \mathbf{A}(\mathbf{w} - \mathbf{a})$$

Assume no special properties for matrices $\mathbf{A}$ and $\mathbf{B}$.

Matrix Derivations: Gradient of MSE

Applying the identities to each term of $E(\mathbf{w})$:

$$\frac{\partial E}{\partial \mathbf{w}} = \frac{1}{n} \frac{\partial}{\partial \mathbf{w}} (\mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} + \mathbf{y}^\top \mathbf{y} - 2\mathbf{w}^\top \mathbf{X}^\top \mathbf{y})$$

Differentiating term-by-term:

$$\frac{\partial}{\partial \mathbf{w}} (\mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w}) = 2\mathbf{X}^\top \mathbf{X} \mathbf{w}$$

$$\frac{\partial}{\partial \mathbf{w}} (\mathbf{y}^\top \mathbf{y}) = \mathbf{0}$$

$$\frac{\partial}{\partial \mathbf{w}} (\mathbf{w}^\top \mathbf{X}^\top \mathbf{y}) = \mathbf{X}^\top \mathbf{y}$$

Setting the gradient to zero:

$$\frac{\partial E}{\partial \mathbf{w}} = \frac{2}{n} \mathbf{X}^\top \mathbf{X} \mathbf{w} - \frac{2}{n} \mathbf{X}^\top \mathbf{y} = \mathbf{0} \implies \mathbf{X}^\top \mathbf{X} \mathbf{w} = \mathbf{X}^\top \mathbf{y}$$

Matrix Derivations: The Normal Equations

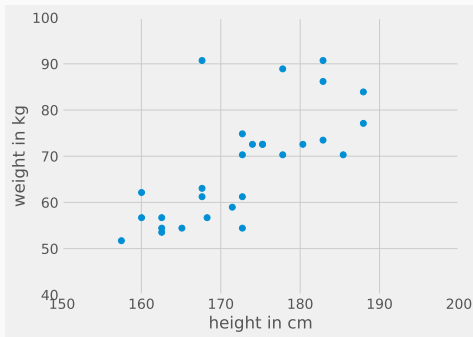
This linear system is known as the **normal equations**:

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

Notice that this is identical to the system derived entry-by-entry on slide 10.

Curve Fitting: Affine Model Example

Let's return to our height vs. weight problem:



Curve Fitting: Deriving the LS Solution

The affine model is $y = w_0 + w_1x$. The design matrix is:

$$\mathbf{X} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}$$

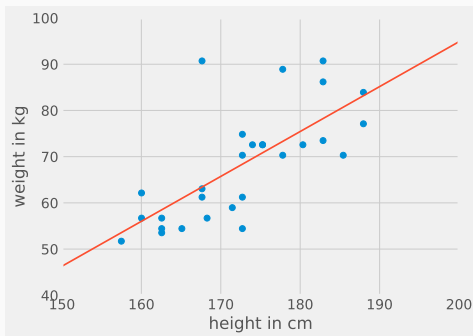
The normal equation components evaluate to:

$$\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} \sum_{i=1}^n 1 & \sum_{i=1}^n x_i \\ \sum_{i=1}^n x_i & \sum_{i=1}^n x_i^2 \end{pmatrix}, \quad \mathbf{X}^\top \mathbf{y} = \begin{pmatrix} \sum_{i=1}^n y_i \\ \sum_{i=1}^n x_i y_i \end{pmatrix}$$

Provided $\mathbf{X}$ has full column rank ($\mathbf{X}^\top \mathbf{X}$ is invertible), the LS estimate is:

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} = \begin{pmatrix} \sum_{i=1}^n 1 & \sum_{i=1}^n x_i \\ \sum_{i=1}^n x_i & \sum_{i=1}^n x_i^2 \end{pmatrix}^{-1} \begin{pmatrix} \sum_{i=1}^n y_i \\ \sum_{i=1}^n x_i y_i \end{pmatrix}$$

Curve Fitting: Fitted Affine Model



Numerically, we obtain $\hat{\mathbf{w}} = \begin{pmatrix} -99.5 \\ 0.972 \end{pmatrix}$, yielding:

$$\text{weight} = \text{height} \times 0.972 - 99.5$$

Linear Models: Basis Transformations

A linear model is not limited to straight lines or hyperplanes. Consider:

$$y = w_0 + w_1x + w_2x^2 + w_3x^3$$

This is still a **linear model** because y is linear in the unknown parameters $\mathbf{w}$. We have simply mapped input x to feature vector $\mathbf{x} = (1, x, x^2, x^3)^\top$.

Similarly:

$$y = w_0 + w_1 \cos(2\pi x) + w_2 \sin(2\pi x)$$

is linear in parameters with $\mathbf{x} = (1, \cos(2\pi x), \sin(2\pi x))^\top$.

Conversely, $y = w_0^2 + x$ is **not** linear because it is non-linear in parameter w_0 .

Linear Models: General Formulation

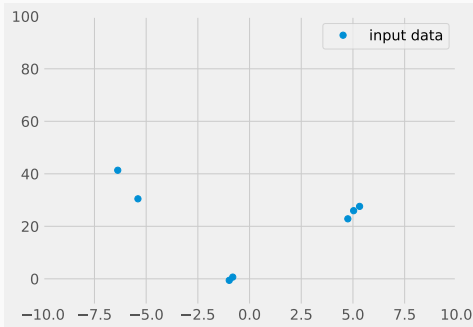
A model is linear in the parameters if it can be written as:

$$y = f(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^p w_j \phi_j(\mathbf{x})$$

where basis functions $\phi_j(\mathbf{x})$ can be non-linear, but remain strictly independent of $\mathbf{w}$.

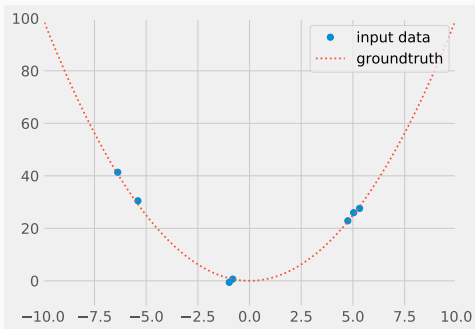
Curve Fitting: Polynomial Example

Fitting non-linear curves using linear least squares:



Curve Fitting: True Latent Model

The ground-truth process generating the data is $y = w_0 + w_1x + w_2x^2$:



Curve Fitting: Polynomial Normal Equations

For model $y = w_0 + w_1x + w_2x^2$, the polynomial design matrix is:

$$\mathbf{X} = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 \end{pmatrix}$$

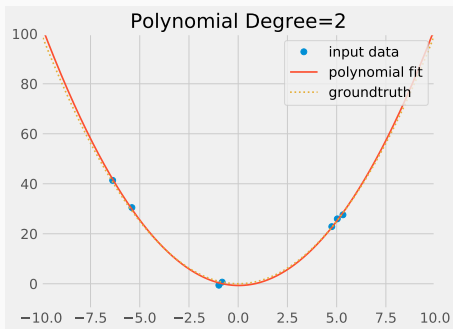
$$\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} \sum 1 & \sum x_i & \sum x_i^2 \\ \sum x_i & \sum x_i^2 & \sum x_i^3 \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 \end{pmatrix}, \quad \mathbf{X}^\top \mathbf{y} = \begin{pmatrix} \sum y_i \\ \sum x_i y_i \\ \sum x_i^2 y_i \end{pmatrix}$$

The LS parameter estimate remains:

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Curve Fitting: Degree-2 Polynomial Fit

Estimated second-order polynomial:

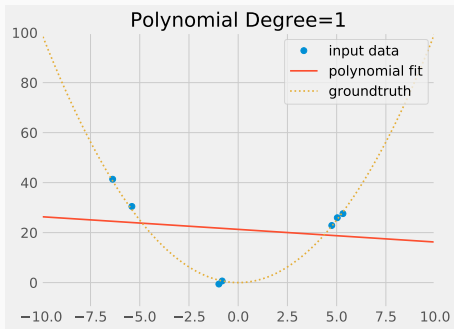


MSE: 4.38e-01

Underfitting: Model of Lower Degree

What happens when we fit the quadratic data with a first-order model

$$y = w_0 + w_1x$$



MSE: 2.02e+02

Underfitting: Diagnosis

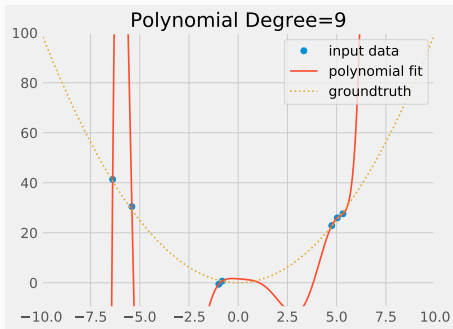
This pathology is called **underfitting**.

How do you know that you are underfitting?

You are underfitting when the model is too restrictive to capture the structure in the data: the training error cannot be reduced to a satisfactory level.

Overfitting: High-Degree Polynomial

Let's try a 9th-degree polynomial: $y = w_0 + w_1x + \dots + w_9x^9$



MSE: 7.59e-06

Overfitting: Generalisation Error

Although training error is near zero ($\text{MSE} = 7.59 \times 10^{-6}$), the curve oscillates wildly between sample points.

This pathology is called **overfitting**.

Given enough free parameters, a model will fit training observations arbitrarily closely, including noise, but fail to generalise to unseen test data.

Overfitting: Diagnosis

How do you know that you are overfitting?

You know that you are overfitting when the training error is low, but the prediction error on new, unseen data is large.

Combatting Overfitting: Model Complexity

First, check whether your model class is unnecessarily complex: Use a simpler model and ensure it does not underfit.

e.g. Do not use degree 9 when underlying phenomenon is degree 2.

Combatting Overfitting: Data Scale

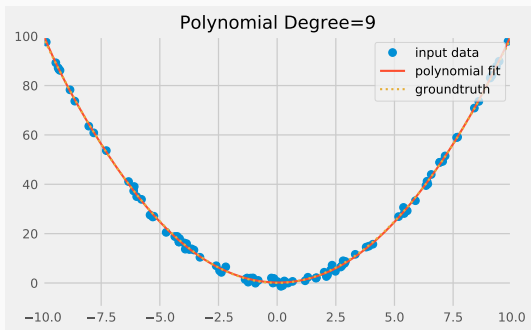
Often the model class is reasonable, but there are too few observations to constrain all coefficients: **Get more training data**.

e.g. 5 observations cannot reliably constrain a 9th-degree polynomial.

Combatting Overfitting: Parameter Shrinkage

Using more data is always a good idea. It allows to use models that are more complex than necessary.

With sufficient samples, irrelevant coefficients naturally shrink towards zero:



With dense data: $\hat{w}_9 = -1.83 \times 10^{-8}$

Combatting Overfitting: The Data Dilemma

But what if you can't get enough data?

Combatting Overfitting: The Data Dilemma

But what if you can't get enough data?

- > Get more data.

Combatting Overfitting: The Data Dilemma

But what if you can't get enough data?

> Get more data.

But what if really can't?

Combatting Overfitting: The Data Dilemma

But what if you can't get enough data?

- > Get more data.

But what if really can't?

- > I don't care. Come back with more data.

Combatting Overfitting: The Data Dilemma

But what if you can't get enough data?

- > Get more data.

But what if really can't?

- > I don't care. Come back with more data.

... seriously?

Combatting Overfitting: The Data Dilemma

But what if you can't get enough data?

> Get more data.

But what if really can't?

> I don't care. Come back with more data.

... seriously?

> I am serious. Go and get more data.

Combatting Overfitting: The Data Dilemma

But what if you can't get enough data?

> Get more data.

But what if really can't?

> I don't care. Come back with more data.

... seriously?

> I am serious. Go and get more data.

Please?

Combatting Overfitting: The Data Dilemma

But what if you can't get enough data?

> Get more data.

But what if really can't?

> I don't care. Come back with more data.

... seriously?

> I am serious. Go and get more data.

Please?

> Fine. One alternative solution to more data is to use **regularisation**.

Tikhonov Regularisation

In Least Squares, standard ℓ_2 shrinkage is known as **Tikhonov regularisation** (or **Ridge regression**).

Instead of minimising $\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$, we add a parameter penalty:

$$E(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \alpha \|\mathbf{w}\|^2$$

This penalises large coefficients, guarantees invertibility of the linear system, and admits a closed-form solution:

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X} + \alpha \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

Note: In practice, the intercept w_0 is left unregularised by setting the top-left diagonal entry of the identity matrix $\mathbf{I}$ to 0.

Regularisation: Intuition

Tikhonov regularisation pulls weights towards zero when evidence in the data is insufficient to justify large values.

It encodes a prior assumption: without evidence to the contrary, simpler, smaller weights are more plausible.

e.g. It is much more plausible that:

$$\text{weight} = \text{height} \times 0.972 - 99.5$$

than:

$$\text{weight} = \text{height} \times 10^{10} - 10^{20}$$

Regularisation: Trade-offs

Regularisation avoids severe instability when evaluating predictions outside dense regions of training inputs.

However, this stability introduces estimation bias.

Whenever possible, prefer collecting sufficient samples so that $\mathbf{X}^T\mathbf{X}$ is comfortably conditioned on its own.

So get plenty of data!

Maximum Likelihood

Probabilistic Perspective

Carl Friedrich Gauss connected least squares directly to probability theory and the normal distribution.

Maximum Likelihood: Noise Model

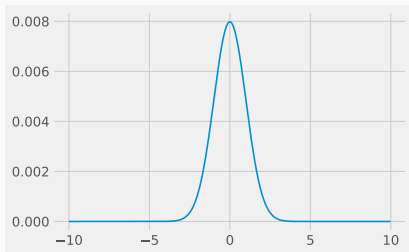
Recall our matrix linear model:

$$\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\varepsilon}$$

Assume observation errors ε_i are independent and identically distributed (i.i.d.) Gaussian random variables:

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

$$p(\varepsilon) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\varepsilon^2}{2\sigma^2}\right)$$



Maximum Likelihood: Joint Likelihood

Given $\mathbf{x}_i$ and parameters $\mathbf{w}$, observation y_i has conditional density:

$$p(y_i \mid \mathbf{x}_i, \mathbf{w}) = p(\varepsilon_i = y_i - \mathbf{x}_i^\top \mathbf{w}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \mathbf{x}_i^\top \mathbf{w})^2}{2\sigma^2}\right)$$

Assuming independence across samples, the joint likelihood is:

$$\begin{aligned} p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) &= \prod_{i=1}^n p(y_i \mid \mathbf{x}_i, \mathbf{w}) \\ &= \left(\frac{1}{\sqrt{2\pi\sigma^2}}\right)^n \exp\left(-\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \mathbf{w})^2\right) \end{aligned}$$

Maximum Likelihood: Derivation

We seek the parameter vector that maximises the likelihood:

$$\hat{\mathbf{w}}_{\text{ML}} = \arg \max_{\mathbf{w}} p(\mathbf{y} \mid \mathbf{X}, \mathbf{w})$$

Equivalently, minimise the negative log-likelihood:

$$\begin{aligned}\hat{\mathbf{w}}_{\text{ML}} &= \arg \min_{\mathbf{w}} [-\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w})] \\ &= \arg \min_{\mathbf{w}} \left[\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \mathbf{w})^2 + n \log (\sqrt{2\pi\sigma^2}) \right] \\ &= \arg \min_{\mathbf{w}} \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \mathbf{w})^2\end{aligned}$$

Maximum Likelihood: Equivalence

Minimising the sum of squared residuals is mathematically identical to maximum likelihood estimation under additive i.i.d. Gaussian noise.

Historical Notes

Origins of Least Squares

Least squares originated from celestial mechanics and planetary orbit calculations. Published first by Adrien-Marie **Legendre** (1805), Carl Friedrich **Gauss** (1809) claimed priority based on earlier working use dating back to 1795.

APPENDICE.

Sur la Méthode des moindres quarrés.

DANS la plupart des questions où il s'agit de tirer des mesures données par l'observation, les résultats les plus exacts qu'elles peuvent offrir, on est presque toujours conduit à un système d'équations de la forme

$$E = a + bx + cy + fz + \&c.$$

dans lesquelles $a, b, c, f, \&c.$ sont des coefficients connus, qui varient d'une équation à l'autre, et $x, y, z, \&c.$ sont des inconnues qu'il faut déterminer par la condition que la valeur de E se réduise, pour chaque équation, à une quantité ou nulle ou très-petite.

Legendre (1805), *Nouvelles méthodes pour la détermination des orbites des comètes.*

Origins of Regression: Francis Galton

The word **Regression** originates from Francis Galton's paper: *Regression towards mediocrity in hereditary stature* (1886).

Galton compared height distributions between parents and their offspring. Using least squares, he observed that parents at extreme percentiles produced offspring closer to the population mean.

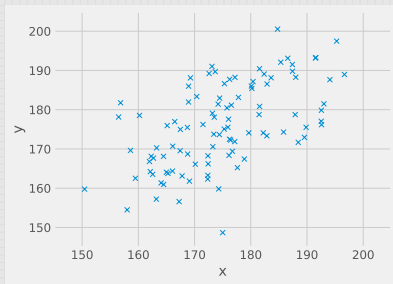
Hence the term **regression towards the mean**.

However, interpreting this as a biological convergence is a statistical artifact of errors-in-variables.

Origins of Regression: Latent Variables

Scatter plot of parent height x vs. offspring height $y \longrightarrow$

The parent height measurement is a noisy proxy for latent genetic factors.



Instead of observing the clean feature x , we observe $x' = x + \nu$. The underlying relationship is:

$$\begin{aligned}y &= w_0 + w_1 x + \varepsilon \\ &= w_0 + w_1 (x' - \nu) + \varepsilon\end{aligned}$$

where ε is model noise and ν is feature measurement noise.

Origins of Regression: Noisy Features

What is the impact of noisy features $x' = x + \nu$ on the normal equations?

The observed design matrix $\mathbf{X}'$ gives:

$$\mathbf{X}'^\top \mathbf{X}' = \begin{pmatrix} \sum 1 & \sum (x_i + \nu_i) \\ \sum (x_i + \nu_i) & \sum (x_i + \nu_i)^2 \end{pmatrix}, \quad \mathbf{X}'^\top \mathbf{y} = \begin{pmatrix} \sum y_i \\ \sum (x_i + \nu_i) y_i \end{pmatrix}$$

Assuming ν is zero-mean and independent of x and y , as $n \rightarrow \infty$:

$$\frac{1}{n} \sum \nu_i \rightarrow 0, \quad \frac{1}{n} \sum \nu_i x_i \rightarrow 0, \quad \frac{1}{n} \sum \nu_i y_i \rightarrow 0, \quad \frac{1}{n} \sum \nu_i^2 \rightarrow \sigma_\nu^2$$

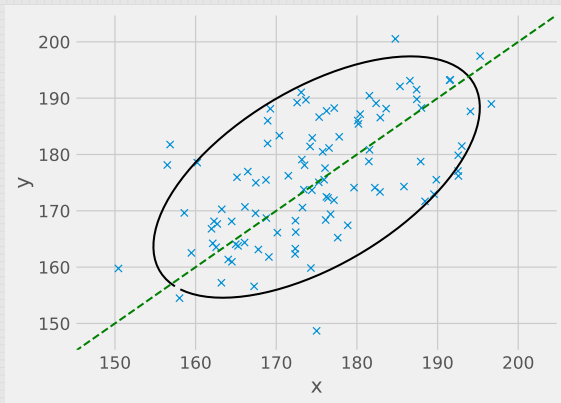
Thus $\mathbf{X}'^\top \mathbf{y} \approx \mathbf{X}^\top \mathbf{y}$, and the least squares estimate satisfies:

$$\hat{\mathbf{w}} = (\mathbf{X}'^\top \mathbf{X}')^{-1} \mathbf{X}'^\top \mathbf{y} \approx \left(\mathbf{X}^\top \mathbf{X} + n \begin{pmatrix} 0 & 0 \\ 0 & \sigma_\nu^2 \end{pmatrix} \right)^{-1} \mathbf{X}^\top \mathbf{y}$$

Noise variance σ_ν^2 introduces an implicit diagonal regulariser, attenuating $\hat{w}_1$ towards 0 (known as [attenuation bias](#)).

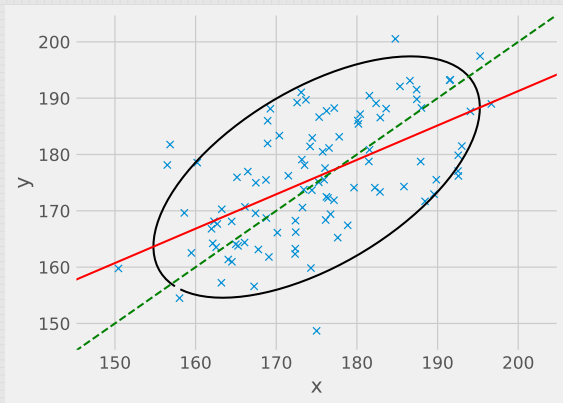
Regression to the Mean: Ground Truth

True generative slope without measurement noise:



Regression to the Mean: Attenuated LS Fit

Attenuated slope obtained with standard LS under noisy inputs:



Conclusion: Measurement Noise

If inputs are noisy, standard least squares biases the estimated slope towards zero.

Standard linear regression assumes observations satisfy $y = \mathbf{x}^T \mathbf{w} + \varepsilon$, where input regressors $\mathbf{x}$ are known without error.

When features contain measurement noise ($\mathbf{x}' = \mathbf{x} + \boldsymbol{\nu}$), regressors correlate with the effective disturbance, violating ordinary least squares assumptions.

Takeaways

We start from a dataset of n observations $(\mathbf{x}_i, y_i)_{i=1}^n$. Each observation possesses p original features $(x_{i1}, \dots, x_{ip})$.

The feature vector can be augmented: we prepend a constant $x_{i0} = 1$ to accommodate the intercept ($\mathbf{x}_i = (1, x_{i1}, \dots, x_{ip})^\top$), or include non-linear transformations such as $x_{ik} = x_{i1}^2$.

The core structural assumption is that targets are linear in parameters:

$$y_i = \mathbf{x}_i^\top \mathbf{w} + \varepsilon_i$$

We define a **loss function** to quantify fitting error: in Least Squares, this is the **Mean Squared Error** (MSE).

The closed-form solution $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ is derived via the normal equations and coincides with the maximum likelihood estimator under i.i.d. Gaussian error.

Underfitting is addressed by enriching the feature representation; overfitting is addressed by collecting more observations or applying regularisation.